

電腦會聽你的指令嗎？

Will the computer follow your instructions?

```
message = "Hello, AI!"  
print(message)
```

猜猜看：螢幕會顯示甚麼？

Guess: what will appear on the screen?

先看老師示範，開啟自己的副本後再試

Watch the teacher first, then repeat in your own copy

換一句話，試試看

Change the greeting and try again

```
message = "Hello, future robot!"  
print(message)
```

改引號內的文字，再執行一次

Change the text inside the quotes, then run again

先把文字存成 message，再用 print 顯示

Store the text as message, then display it with print

今天的挑戰：讓電腦找一條線

Today's challenge: let the computer find the line

上堂：你用眼睛觀察，畫一條合適的直線

Last time: you looked at the dots and drew a line

今堂：用 Python 執行、改數據、作預測

Today: use Python to run code, change data and predict

目標是看懂變化，無需背下每一行程式

Notice what changes. You do not need to memorise every line

Python、Notebook、Colab 是甚麼？

What are Python, a notebook and Colab?

Python：給電腦指令的程式語言

Python: the programming language for instructions

Jupyter Notebook：把文字、程式和結果放在一起

Jupyter Notebook: text, code and results together

Colab：在瀏覽器使用 Notebook，程式在雲端執行

Colab: use a notebook in a browser; code runs on a cloud computer

你的第一本「實驗簿」

Your first experiment notebook

開啟老師的連結，儲存自己的副本
Open the teacher's link and save your own copy

文字格是說明；程式格是可執行的指令
Text cells explain; code cells contain instructions to run

按程式格左邊的 ►，每次執行一格。結果顯示在下方
Click  on the left of a code cell. Run one cell at a time

資源 / Resources: gordany.github.io/f2-linear-regression/workshop-2/

數據也要排好隊

The data needs matching pairs

```
hours    = [0, 1, 2, 3, 4, 5, 6]  
scores   = [53, 46, 59, 52, 63, 54, 65]
```

第 2 對：(1 小時, 46 分)。兩個列表要有相同數量

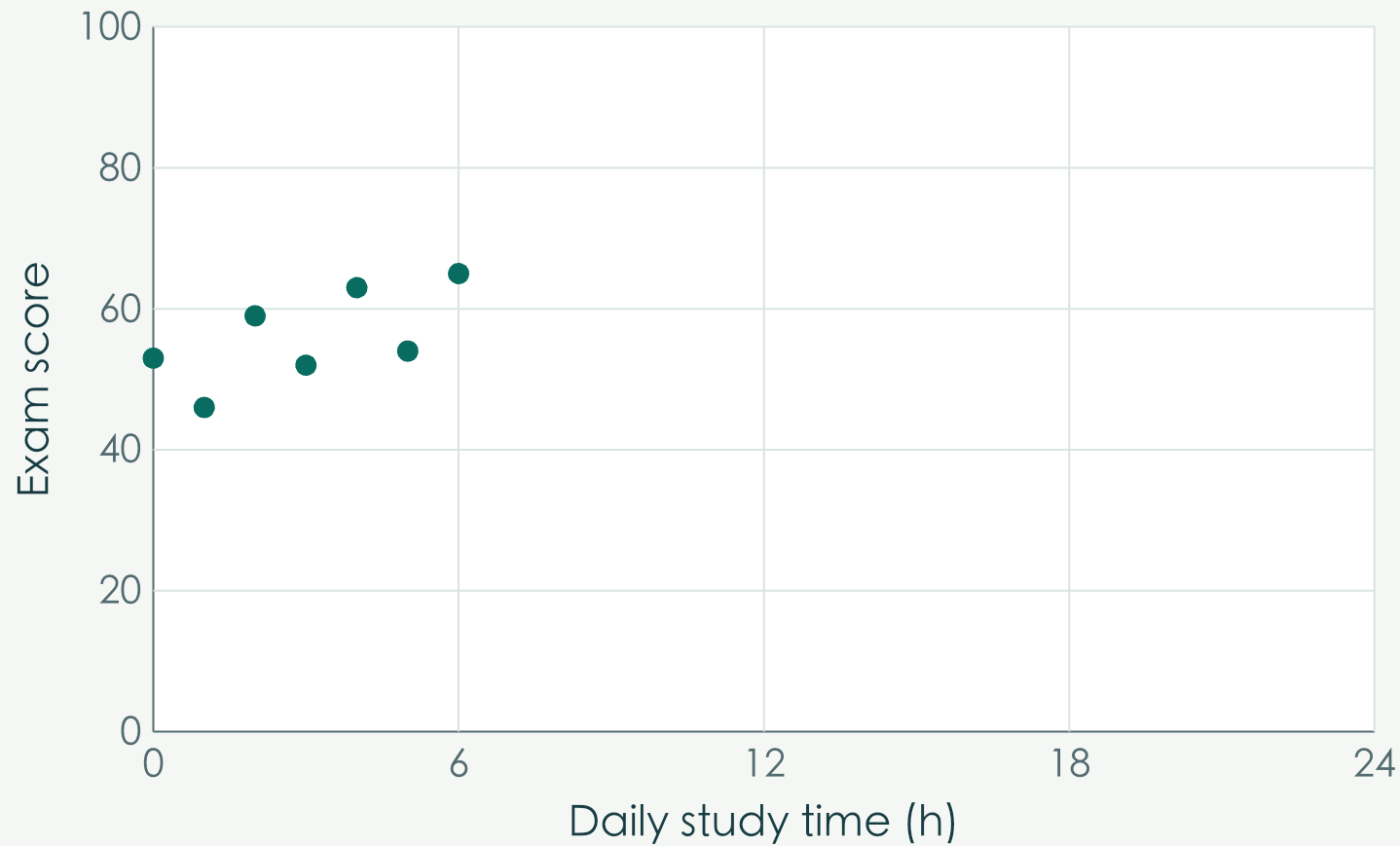
The second pair is (1 hour, 46 marks). Both lists need equal lengths

先看散點，猜每天 3.5 小時會得多少分

Look at the dots first. Guess the score for 3.5 hours a day

先猜一猜，暫時不揭曉

Make a guess before the reveal



3.5 小時，大約幾分？

About what score at 3.5 hours?

先寫答案，再看模型

Write your guess before the model

fit：讓工具從例子找直線

fit: let the tool learn a line from examples

```
model = LinearRegression()  
model.fit(X, y)
```

X 是溫習時間資料，y 是相應成績

X holds the study hours; y holds the matching scores

直線概括整體趨勢，不必穿過每一點

The line summarises the trend; it need not pass through every dot

predict : 請模型猜一個新情況
predict: ask the model about a new case

```
study_hours = 3.5  
model.predict([[study_hours]])
```

只改 study_hours : 試 3.5 , 再試 10

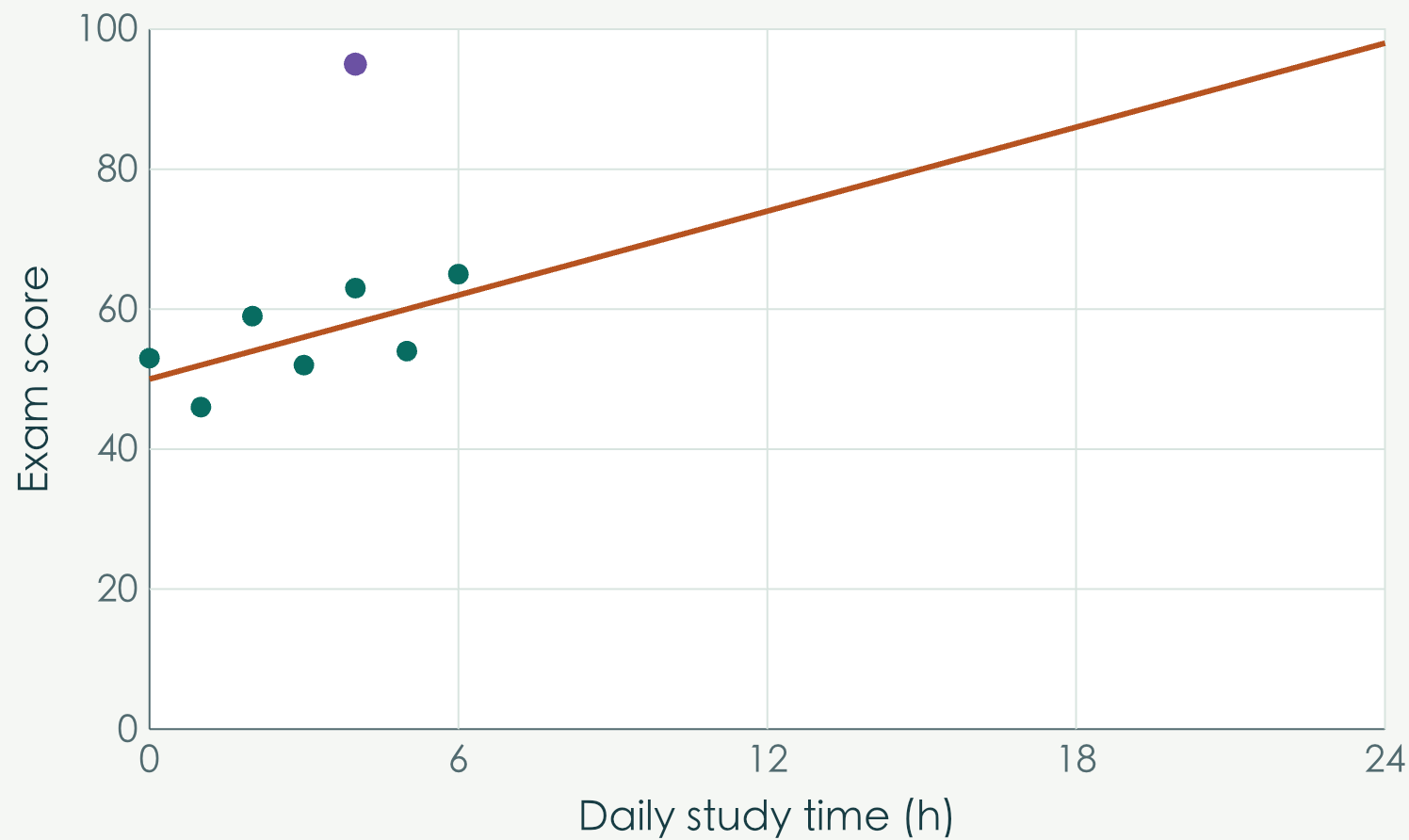
Change only study_hours: try 3.5, then 10

預測是估計。10 小時超出原來 0–6 小時的觀察範圍

A prediction is an estimate. 10 hours is outside the observed 0–6 range

加入一個新數據點

Add a new data point



```
new_point=[4,95]
```

先猜，再執行

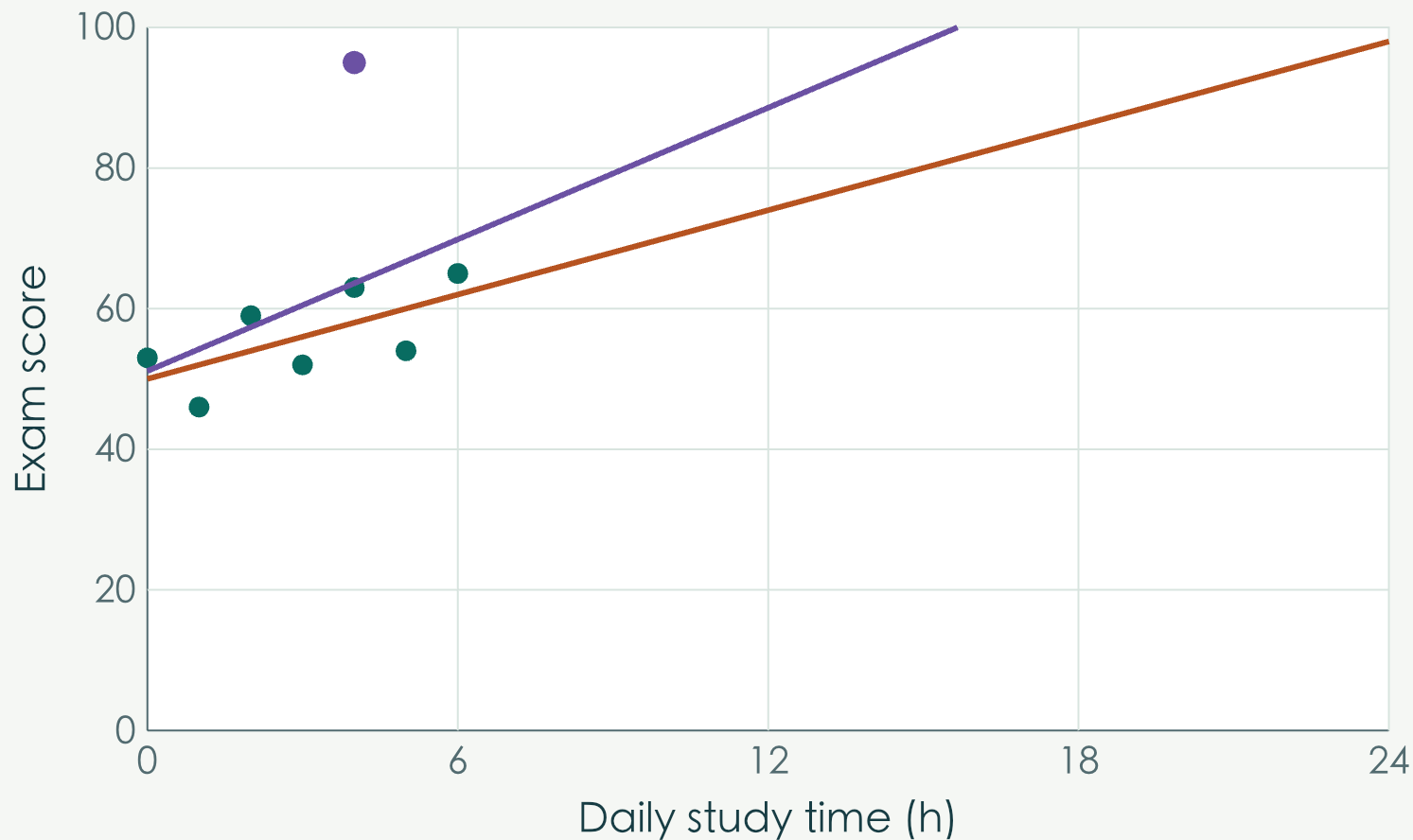
Predict, then run

直線會怎樣變？

How might the line change?

揭曉：原來和加點後的直線

Reveal: compare the two lines



`new_point=[4,95]`

3.5 小時的預測 / At 3.5 hours

原來約 57 分

Before: about 57 marks

加點後約 62 分

After: about 62 marks

橙：原來 / Orange: before 紫：加點後 / Purple: after

這和 AI 有甚麼關係？

How does this connect to AI?

我們提供例子，工具從例子建立模型

We provide examples; the tool fits a model from them

改變數據，模型的預測可能改變

Change the data, and predictions may change

程式能執行，不代表模型的答案一定正確

Working code does not guarantee a correct prediction

分享你的觀察

Share your observations

加入新點後，直線或預測有甚麼改變？

How did the line or prediction change after adding the point?

3.5 小時和 10 小時的預測，哪個較值得信賴？

Which prediction is more trustworthy: 3.5 or 10 hours?

下一堂：讓電腦猜「是哪一類」，而不是「幾多分」

Next time: predict which class, instead of how many marks